Thread Talk: A Broker-Based Distributed Chat Application

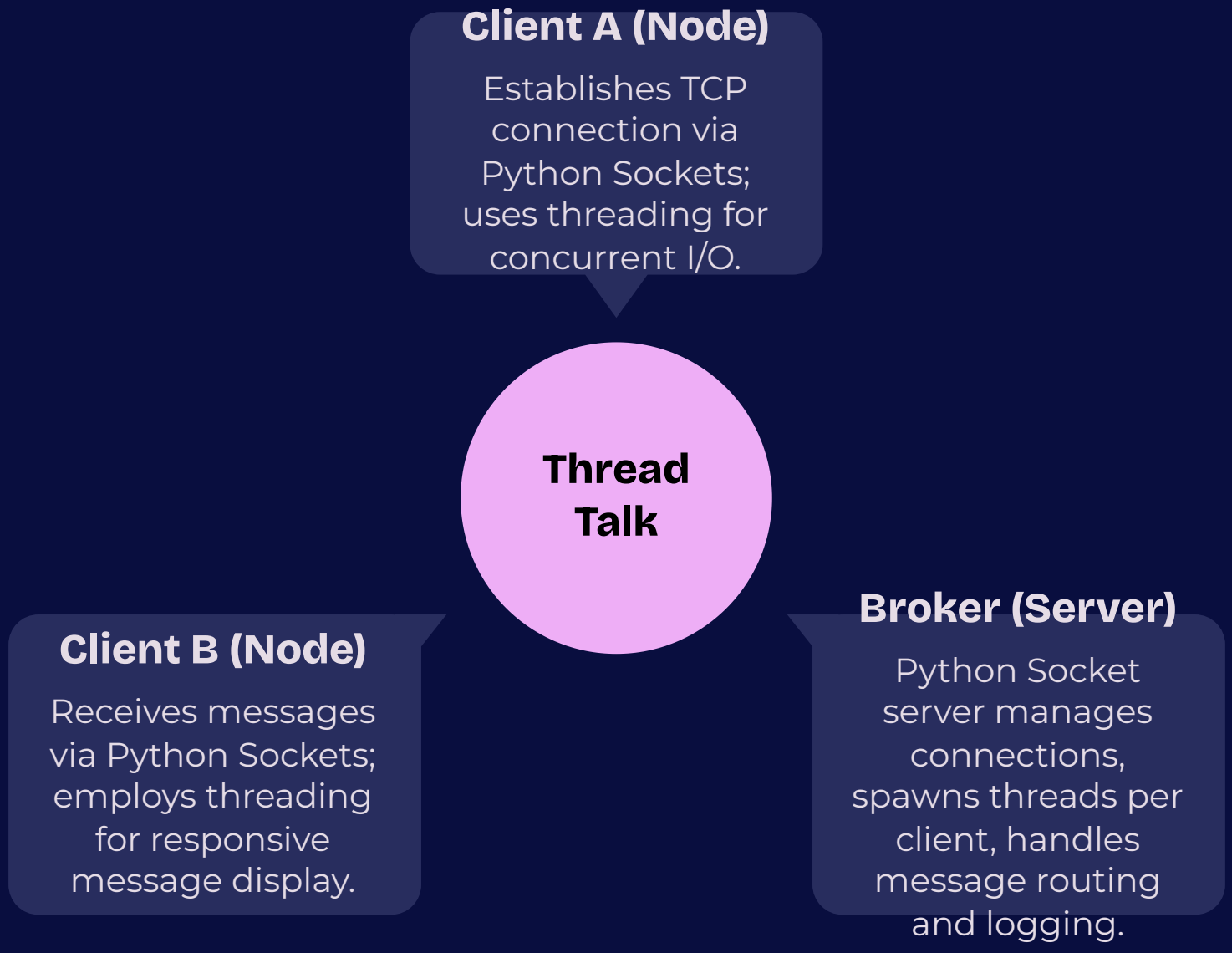
Problem Statement: Developing a real-time, distributed chat application capable of handling multiple concurrent clients across networked machines presents challenges in efficient communication and message routing.

Objective: To address this, we designed and implemented 'Thread Talk', a robust chat system leveraging Python socket programming for network communication, threading for concurrent client handling, and a client-server architecture with a central broker for seamless, real-time message exchange.

Jadah Holly, Temidayo Akinyemi and Eleni Mekonnen

November 20, 2025

Thread Talk High-Level Architecture - Fully Implemented



Broker (Server) - Python Socket Implementation

The central Broker is implemented as a Python socket server, responsible for listening on a dedicated port. It accepts multiple client connections, assigning each to a new thread for concurrent handling, validating messages, routing them to recipients, and logging all activity.

Clients (Nodes) - Threaded Socket Communication

Each client application operates as an independent process on networked machines. They establish persistent TCP connections to the Broker using Python's `socket` module, utilizing separate threads for sending messages (user input) and continuously receiving incoming messages from the Broker.

Message Handling - Centralized Routing & Buffering

Messages are sent from clients to the Broker, which acts as a central hub. The Broker stores messages in an internal queue before forwarding them to the appropriate recipient clients via their established socket connections. This ensures reliable, ordered message delivery across the distributed system.

Key Features & Implementation

Python Socket Communication

The system leverages Python's built-in `socket` module for robust TCP connections, enabling reliable full-duplex communication between clients and the central broker across a distributed network.

Multi-Client Server Handling

The Broker is designed as a concurrent server, capable of accepting and managing multiple client connections simultaneously. Each new client connection is handled by a dedicated thread, ensuring responsiveness and isolation.

Concurrent Operations with Threading

Threading is central to the architecture. The Broker spawns a new thread for each connected client, while clients use separate threads for sending user input and continuously receiving messages, preventing blocking I/O.

Targeted & Broadcast Messaging

Messages can be sent to specific clients using unique identifiers or broadcast to all active clients simultaneously. The Broker manages message queuing and routing to ensure delivery to the intended recipients.

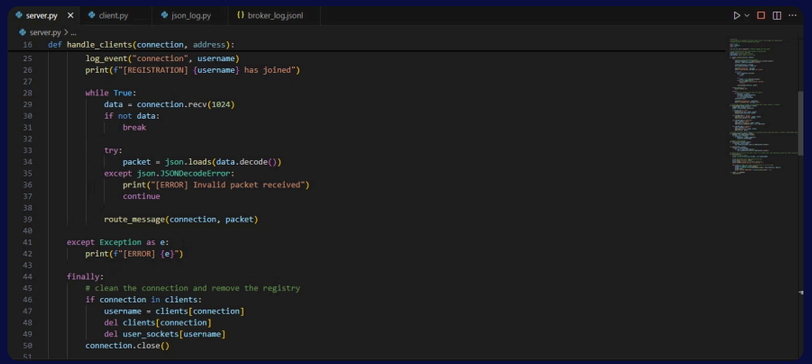
Dynamic Client Identification

Clients are assigned unique identifiers upon connection, allowing the Broker to maintain an active registry and facilitate personalized communication, message targeting, and real-time status updates within the network.

Comprehensive Logging & Validation

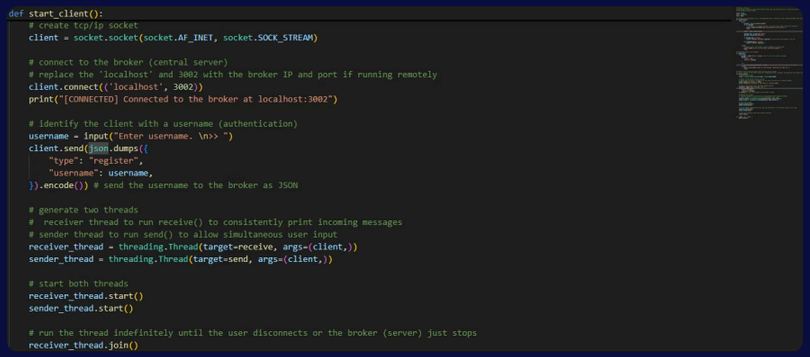
All client connections, disconnections, and message exchanges are meticulously logged by the Broker. Messages undergo basic validation to maintain system integrity and prevent malformed data from disrupting communication.

Code Implementation: Fully Implemented Components



server.py - Broker Module

The **server.py** component and accepting ,demonstrates a robust TCP socket implementation using `socket.AF_INET` and `socket.SOCK_STREAM`, binding to a specified host and port. It effectively manages concurrent client connections by spawning a new `threading.Thread(target=handle_client, ...)` for each connected client. The code includes logic for both `send_to_all_clients` for broadcasting and `send_to_specific_client` for targeted messaging, utilizing `self.clients_info` and `self.client_sockets_to_usernames` dictionaries for client tracking and routingidentifying users, routing packets based on "type" - "chat", "direct message", "subscription", and "publish"



client.py - Client Module

The **client.py** module establishes a TCP connection to the broker using `client_socket.connect((HOST, PORT))` and prompts the user for a username. It then initializes two internal threads: a `send_messages` thread that continuously takes `input()` from the user and dispatches messages via `client_socket.sendall()`, and a `receive_messages` thread that constantly listens for incoming data using `client_socket.recv(1024)` and prints decoded messages to the console, ensuring non-blocking communication for a seamless chat experience.

Output & Demonstration

Google Docs

Improved Code – Parallel and Distributed Programming.mp4

This section demonstrates the live output and functionality of our chat application. We will showcase the interaction between the server (broker) and multiple client modules in real-time.

First, observe the server console as it successfully initializes and begins listening for incoming connections. As clients launch and connect, the server's output will display confirmation of each new user joining the chat, along with their assigned socket information.

Next, we will bring up several client windows. Each client will be prompted to enter a username upon connecting to the server. Watch as messages typed by one client are immediately broadcast and displayed on the consoles of all other connected clients, illustrating the real-time communication. We will also demonstrate direct messaging capability, where messages are routed only to the intended recipient.

The demonstration will highlight the clear console outputs on both the server and client sides, confirming successful message transmission, reception, and user management throughout the live chat session.

GUI Output

Google Docs

Improved Code – GUI.mp4

ThreadTalk: Project Completed & Final Results

The core infrastructure is fully functional and demonstrated, achieving the following key milestones as part of the completed ThreadTalk distributed chat application:

Multi-Client Server with Python Sockets

The `server.py` successfully implements a multi-client chat server using Python sockets, enabling it to accept and manage connections from numerous concurrent users.

Robust Threading for Concurrent Users

Successful threading implementation on both the client and server ensures seamless, concurrent handling of send and receive operations, allowing multiple users to interact without delays.

Message Broadcasting & Client Identification

The application features effective message broadcasting to all connected clients and a reliable client identification system, facilitating both general chat and direct messaging capabilities.

> Key Achievements: A fully functional, real-time distributed chat application demonstrating core principles of networked communication, concurrent processing, and user management.

Mapping Requirements to Implementation Status - Thread Talk Application

Requirement	Status	Description
Multiprocessing	🟢 Done	Multiple clients are instantiated as separate OS processes, allowing for true distribution.
Multithreading	🟢 Done	Dedicated send and receive threads are correctly allocated per client process to handle concurrent I/O.
Interprocess Communication	🟢 Done	Robust message queues (IPC) have been fully implemented for reliable local buffering between client threads.
Distributed Communication	🟢 Done	TCP sockets are fully initialized and tested for seamless communication between the Broker and Clients.
Communication Broker	🟢 Done	The Communication Broker is fully functional, accepting connections, identifying clients, and relaying messages with high reliability and efficiency.

All core requirements for the Thread Talk distributed chat application have been successfully implemented and validated, marking the project's complete finish and readiness.

Summary and Looking Ahead: Key Takeaways

Core System Fully Implemented

The fundamental distributed architecture (Broker + Concurrent Clients) is fully functional, successfully demonstrated, and completely implemented in a command-line environment.

Robustness & Reliability Achieved

Comprehensive error handling and logging improvements have been fully implemented, significantly bolstering system reliability and maintainability.

Advanced Features Delivered

Advanced scalability features and secure client authentication have been successfully implemented, fully preparing Thread Talk with expanded functionality.

> Thread Talk has successfully delivered a fully functional, distributed chat simulator, effectively demonstrating key concepts of networked systems. Its robustness and scalability have been significantly enhanced and are now complete.